

Unit 4 – LinkedList

CS 201 - Data Structures II

Spring 2024

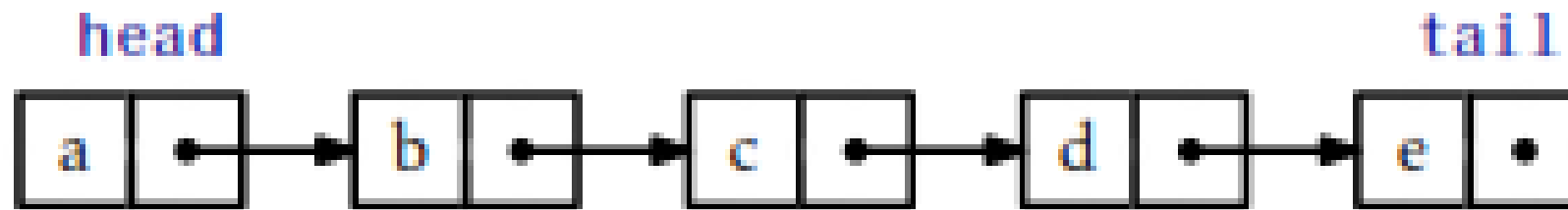
Habib University

Syeda Saleha Raza

Outline

- Motivation
- Singly Linked List – Recap
 - Structure
 - Operations
 - Implementation
- Linked List to implement Stack, Queue, DEQUE interfaces
- Complexity Analysis
- Doubly Linked List

Linked List



Implementation

```
class IntSLLNode {
public:
    IntSLLNode() {
        next = 0;
    }
    IntSLLNode(int el, IntSLLNode *ptr = 0) {
        info = el; next = ptr;
    }
    int info;
    IntSLLNode *next;
};
```

```
class IntSLLList {
public:
    IntSLLList() {
        head = tail = 0;
    }
    ~IntSLLList();
    int isEmpty() {
        return head == 0;
    }
    void addToHead(int);
    void addToTail(int);
    int deleteFromHead(); // delete the head and return its info;
    int deleteFromTail(); // delete the tail and return its info;
    void deleteNode(int);
    bool isInList(int) const;
private:
    IntSLLNode *head, *tail;
};
```

Time Complexity

What is the time complexity of the following operations?

- addtoHead(x)
- addtoTail(x)
- deletefromHead()
- deletefromTail()
- addNode(x,i)
- deleteNode(i)
- isInList(x)

Using LinkedList to implement Stack, Queue, DEQUE

- Fill this table:

Stack	Linked List Operation	Time Complexity
Push	addFromHead	O(1)
Pop	removeFromHead	O(1)
Queue		
Enqueue	addFromTail	O(1)
Dequeue	removeFromHead	O(1)

Doubly Linked List

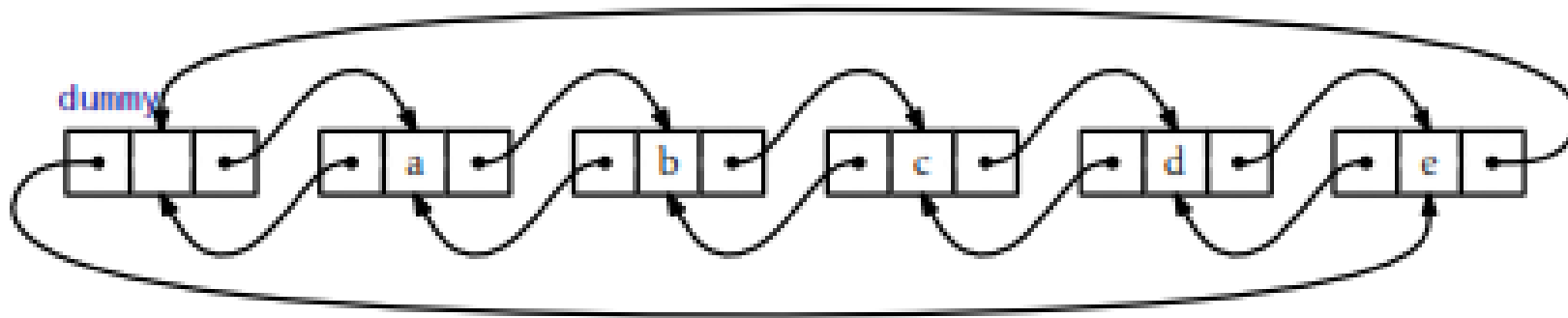


Figure 3.2: A DLL containing a,b,c,d,e.

Doubly Linked List

```
template<class T>
class DLLNode {
public:
    DLLNode() {
        next = prev = 0;
    }
    DLLNode(const T& el, DLLNode *n = 0, DLLNode *p = 0) {
        info = el; next = n; prev = p;
    }
    T info;
    DLLNode *next, *prev;
};
```

```
template<class T>
class DoublyLinkedList {
public:
    DoublyLinkedList() {
        head = tail = 0;
    }
    void addToDLLTail(const T&);
    T deleteFromDLLTail();
    . . . . .
protected:
    DLLNode<T> *head, *tail;
};
```


Having a dummy node

- The dummy node is a node that does not contain any data, but acts as a placeholder so that there are no special nodes.
- Every node has both a **next** and a **prev**, with **dummy** acting as the node that follows the last node in the list and that precedes the first node in the list.
- In this way, the nodes of the list are (doubly-)linked into a cycle,

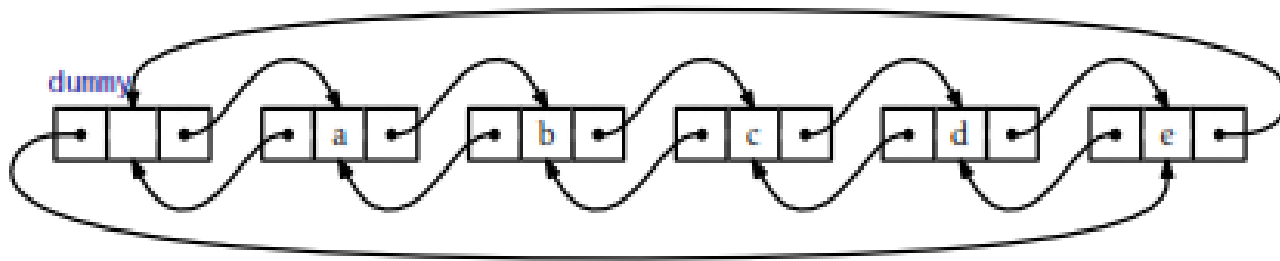


Figure 3.2: A DLL list containing a,b,c,d,e.

DLList Operations

- Addition
- Deletion

Finding a node in a DLList

- Finding the node with a particular index in a DLList is easy:
 - we can either start at the head of the list (*dummy->next*) and work forward,
 - or start at the tail of the list (*dummy->prev*) and work backward.
- *What would be the ime complexity in this case?*

```
                                DLList
Node* getNode(int i) {
    Node* p;
    if (i < n / 2) {
        p = dummy.next;
        for (int j = 0; j < i; j++)
            p = p->next;
    } else {
        p = &dummy;
        for (int j = n; j > i; j--)
            p = p->prev;
    }
    return (p);
}
```

What key issue do we face in Linked Lists?

Next Topic

- Skip List

Resources

- Open Data Structures (pseudocode edition), by Pat Morin. Available online at <http://opendatastructures.org>
- Data Structures and Algorithms in Python, by Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser. 2013. (1st. ed.). Wiley Publishing
- <https://www.khanacademy.org/computing/computer-science/algorithms/asymptotic-notation/a/asymptotic-notation>

Thanks